

Tongji University

决策树 ID3 算法实验报告

于沛可 2451951

数据科学与大数据技术

一．实验目的

1. 掌握决策树算法（ID3）的基本原理

通过实现 ID3 算法，理解决策树基本原理，掌握信息增益、熵的计算、特征选择和数据划分等构建决策树的步骤。

2. 练习使用 Python 实现决策树并进行训练与测试

使用 Python 实现决策树的训练过程：学会数据集的划分，运用交叉验证评估模型算法性能。

3. 应用交叉验证和可视化技术

掌握使用 K 折交叉验证（K=5）来评估模型的泛化能力，并利用图表对实验结果进行可视化展示。

二．实验内容

1. 数据准备：

加载 Car Evaluation 数据集，该数据集包含汽车的六个属性（buying、maint、doors、persons、lug_boot、safety）和一个目标类别（class）。

2. 核心算法实现：

2.1 实现信息熵 (calc_entropy) 和信息增益 (calc_info_gain) 的计算函数。

2.2 实现最佳特征选择函数，基于最大信息增益来选择当前最优的分裂特征。

2.3 实现决策树构建 (create_tree) 函数，采用递归方法生成树结构，并在满足终止条件（数据集纯净或特征用尽）时生成叶节点。

3. 模型评估与验证：

计算模型在划分的训练集和测试集上的分类准确率。实现 5 折交叉验证(cross_validation) 来评估模型的平均性能和稳定性。并使用 Matplotlib 绘制交叉验证中每折准确率和平均准确率的折线图。

三．实验过程

1. 实验环境准备：

确保已安装 Python 环境，并配置必要的依赖库（如 math、csv、collections.Counter、random、matplotlib）。

2. 数据加载与划分

数据加载：使用 load_data()函数加载 CSV 文件，总数据条数为 1728。

```
# 读取 CSV 文件
def load_data(filename):
    data = []
    headers = ['buying','maint','doors','persons','lug_boot','safety','class']
    with open(filename, 'r', encoding='utf-8') as f:
        reader = csv.reader(f)
        for row in reader:
            data.append(row)
    return headers, data
```

将数据集随机打乱并划分为 80%的训练集(1382 条)和 20%的测试集(346 条)。

```
# 训练/测试集划分
random.shuffle(data)
train_size = int(len(data)*0.8)
train_data = data[:train_size]
test_data = data[train_size:]
print(f"训练集大小: {len(train_data)}, 测试集大小: {len(test_data)}")
```

3. 决策树模型构建

3.1 实现信息熵 (calc_entropy) 和信息增益 (calc_info_gain) 的计算函数。

信息熵：衡量数据集的纯度。熵值越大，数据集的混乱程度越高（即类别混杂）。当一个数据集中的所有样本都属于同一类别时，熵为 0，即纯度最高。

计算公式：$\text{Entropy(S)} = -\sum_i p_i \log_2 p_i$

```python
# 计算信息熵
def calc_entropy(data):

    total = len(data)
    if total == 0:
        return 0 # 空数据集的熵为 0

    # 统计类别标签（数据集的最后一列）的出现次数
    label_count = Counter(row[-1] for row in data)
    entropy = 0
    # 遍历每个类别及其对应的计数
    for count in label_count.values():
        p = count / total # 计算当前类别的概率 p_i
        entropy -= p * math.log2(p) # 根据信息熵公式累加
    return entropy
```

信息增益：衡量特征的区分能力。计算公式是：基准熵（父节点的熵）减去划分后的加权平均熵（子节点的熵）。信息增益越大，表示使用该特征划分数据集后，数据集的纯度提升越明显。

```python
# 计算信息增益
def calc_info_gain(data, index):
    # 计算划分前的基准熵 H(S)
    base_entropy = calc_entropy(data)
    total = len(data)
    # 获取该特征的所有唯一取值
    unique_values = set(row[index] for row in data)
    new_entropy = 0
    # 遍历该特征的每个取值，计算划分后的加权平均熵
    for value in unique_values:
        # 1. 划分数据：根据取值 value 得到子数据集 S_v
        sub_data = split_data(data, index, value)
        # 2. 计算权重：子数据集的比例 P(|S_v|/|S|)
        p = len(sub_data) / total
        # 3. 累加加权熵：Σ P(|S_v|/|S|) * H(S_v)
        new_entropy += p * calc_entropy(sub_data)
    # 信息增益 Gain(S, A) = H(S) - H_new
    return base_entropy - new_entropy
```

3.2 实现最佳特征选择函数，基于最大信息增益来选择当前最优的分裂特征。

特征选择：遍历所有可用特征，计算每个特征带来的信息增益，选择信息增益最大的特征作为当前节点的分裂标准。ID3 算法因此也被称为“贪心算法”，因为它在每一步都追求局部最优划分。

```python
# 选择最佳特征
def choose_best_feature(data):
    # 如果只剩类别标签一列，则特征用尽
    if len(data[0]) == 1:
        return -1
    best_gain = 0
    best_index = -1
    # 遍历所有特征（除了最后一列的类别标签）
    for i in range(len(data[0])-1):
        gain = calc_info_gain(data, i) # 计算当前特征的信息增益
        # 寻找信息增益最大的特征
        if gain > best_gain:
            best_gain = gain
            best_index = i
    return best_index
```

3.3 实现决策树构建 (create_tree) 函数

1. 纯度终止：如果当前数据集所有样本类别相同 (class_list.count(...) == len(class_list))，则直接返回该类别作为叶节点。

2. 特征用尽终止：如果所有特征都被用完 (len(data[0]) == 1)，则返回数据集中多数类别作为叶节点（majority_label）。

3. 递归调用：选择最佳特征后，根据该特征的每个取值（如 buying: vhigh, high, med, low）创建一个分支，并在相应的子数据集上递归调用 create_tree 来构建子树。

```python
# 创建决策树
def create_tree(data, labels):

    class_list = [row[-1] for row in data]

    # 终止条件 1：纯度终止
    # 如果所有样本的类别相同，则返回该类别作为叶节点
    if class_list.count(class_list[0]) == len(class_list):
        return class_list[0]

    # 终止条件 2：特征用尽终止
    # 如果数据集只剩下一列（即只剩类别标签），则返回多数类别作为叶节点
    if len(data[0]) == 1:
        return majority_label(data)

    # 递归划分：选择最佳特征
    best_feature = choose_best_feature(data)
    best_label = labels[best_feature]
    # 创建树节点，节点名为最佳特征名
    tree = {best_label: {}}

    # 下一层递归
    feature_values = set(row[best_feature] for row in data)

    # 遍历最佳特征的每个取值，创建分支
    for value in feature_values:
        # 1. 准备子特征名列表：移除当前已用特征
        sub_labels = labels[:]
        sub_labels.pop(best_feature)

        # 2. 准备子数据集：根据取值 value 划分数据
        sub_data = split_data(data, best_feature, value)

        # 3. 递归调用：对子数据集和子特征名列表继续建树
        tree[best_label][value] = create_tree(sub_data, sub_labels)

    return tree
```

通过这个过程，模型构建完成了从数据特征到汽车可接受性类别（unacc, acc, good, v-good）的映射规则。

流程图：

```
graph TD
    A[开始: 调用 create_tree(数据集 S, 特征列表 L)] --> B{检查终止条件};

    subgraph 终止条件判断
        B --> C{条件 1: S 中所有样本类别相同? (纯净终止)};
        C -- 是 --> D[返回叶节点 (类别标签)];
        C -- 否 --> E{条件 2: 特征列表 L 是否为空? (特征用尽终止)};
        E -- 是 --> F[返回叶节点 (S 中多数类别)];
    end

    E -- 否 --> G[计算并选择最佳特征 A (最大信息增益)];

    subgraph 递归划分与调用
        G --> H[创建当前节点: Tree = {A: {}}];
        H --> I{遍历特征 A 的所有取值 V};
        I --> J(划分数据: 得到子数据集 S_V);
        J --> K(移除特征 A: 得到新的特征列表 L_new);
        K --> L[递归调用 create_tree(S_V, L_new)];
        L --> M[将递归结果作为分支 Tree[A][V] 的值];
        M --> I;
        I --> N[所有取值 V 遍历完毕];
    end

    N --> O[返回当前树结构 Tree];

    D --> P[结束: 返回叶节点];
    F --> P;
    O --> P;
```

四．测试样例说明

本实验数据集选自 Kaggle 网站上 Car Evaluation Dataset（https://www.kaggle.com/datasets/elikplim/car-evaluation-data-set）

该数据集结构如下：

实例数量：1728

输入特征：6 维，均为离散型，

特征	具体描述	取值
buying	购买价格	vhigh, high, med, low
maint	维护费用	vhigh, high, med, low
doors	门数量	2, 3, 4, 5more

特征	具体描述	取值
persons	载客容量	2, 4, more
lug_boot	行李箱大小	small, med, big
safety	安全性	low, med, high

输出目标：汽车接受程度，分别为：

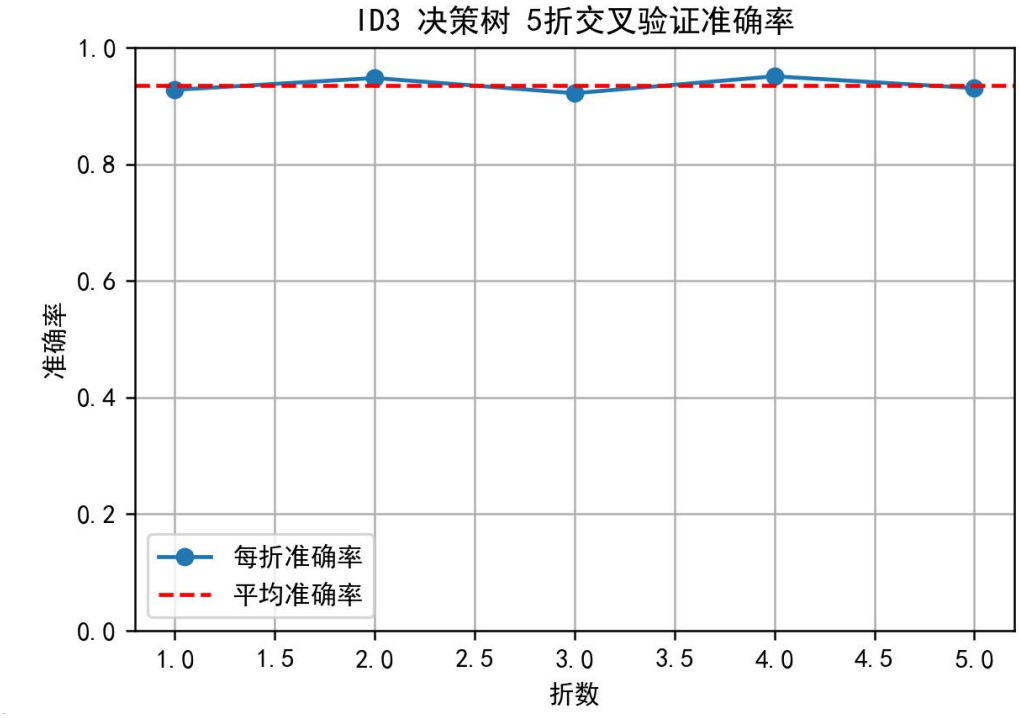
unacc(不可接受)，acc (可接受)，good (好)，v-good (非常好)

五．测试结果及分析



分析：训练集准确率达到 100%，表明构建的决策树能够完美拟合训练数据。测试集准确率为 93.35%，虽然模型在训练集上可能存在过拟合现象（因为数据集较小，且模型没有进行剪枝或其他正则化处理），但其在未见过的数据（测试集）上仍保持了较高的分类准确率，显示出模型具有良好的泛化能力。

为了进一步验证模型性能，使用 5 折交叉验证：将整个数据集随机平均分成 5 个互不重叠的子集，进行 5 次独立的训练和测试：第 1 次使用 Fold 2, 3, 4, 5 作为训练集，使用 Fold 1 作为测试集，得到折 1 准确率；第 2 次使用 Fold 1, 3, 4, 5 作为训练集，使用 Fold 2 作为测试集，得到折 2 准确率……以此类推，共进行 5 次。



观察可得，每折准确率的波动范围较小（最高 0.9362，最低 0.9101），这表明数据集的划分相对均匀，且模型结构对不同的训练子集具有稳定性，从而进一步验证了模型的泛化能力良好。

六．思考

由实验结果可得模型在训练集上的准确率非常之高，达到 1，因此怀疑出现过拟合情况。

因此，引入剪枝机制：

MaxDepth (最大深度)	限制树的最大层数。	防止树生长得过深，从而避免对微小噪声的拟合。

MinSamplesSplit (最小分裂样本数)	只有当节点包含的样本数大于此阈值时，才允许分裂。	避免在只有少量样本的节点上继续分裂，因为这些节点包含的信息量小，噪声占比高。
MinGain (最小信息增益)	只有当最佳特征带来的信息增益大于此阈值时，才允许分裂。	保证分裂是有意义的，避免基于对类别划分帮助微小的特征进行无效分裂。

虽然剪枝是必要的，但引入剪枝参数后会带来一个新的挑战：如何设置这些参数的值？

如果剪枝参数设置得过于严格（剪枝过度），模型会欠拟合；如果剪枝参数设置得过于宽松，则仍然会过拟合。

因此，引入网格搜索方法来寻找最优参数。

结果如下：

```
网格搜索调优完成！
总耗时：13.31 秒
找到的最佳平均交叉验证准确率：0.9380
最佳参数组合：
{'max_depth': 10, 'min_samples_split': 5, 'min_gain': 0.0001}
```

0.9380 较 0.9357 提升仍然较小，提升约 0.23%。说明模型误差的主要来源不再是欠拟合或过拟合，模型的性能已经非常接近该数据集的不可约误差所设定的理论上限。剩余的错误有可能是数据中固有的随机噪声、标签错误或特征无法区分导致的。更换任何算法都难以显著提高准确率。要进一步提升准确率，可能需考虑更换算法，例如随机森林或梯度提升树，以克服单一决策树的固有局限性。